🧪 Lab3 — MIME File Transfer over Router-on-a-Stick with Wireshark Analysis

🎯 Objectives

After completing this lab, you will be able to:

- ✅ Configure Router-on-a-Stick inter-VLAN routing
- ✅ Configure VLANs and trunking on a switch
- ✅ Deploy and run a MIME socket server and client
- ✅ Capture traffic using Wireshark
- ✅ Identify ARP, VLAN tags, TCP handshake, and application data
- ✅ Decode JSON headers in packet payload
- ✅ Troubleshoot network and application failures
- ✅ Document protocol behavior across OSI layers

📝 Lab Report Requirements

Include:

- ✅ Screenshots of configurations
- ✅ Ping results
- ✅ Wireshark capture screenshots
- ✅ JSON header evidence
- ✅ Encapsulation explanation
- ✅ Troubleshooting notes

🔧 Part 1 — Configure Network Devices

Step 1 — Router Configuration

```
None
enable
configure terminal
hostname R1

interface g0/0
no shutdown

interface g0/0.10
encapsulation dot1Q 10
ip address 192.168.10.1 255.255.255.0

interface g0/0.20
encapsulation dot1Q 20
ip address 192.168.20.1 255.255.255.0

end
write memory
```

```
R1>enable
R1#configure terminal

R1(config)#hostname R1
R1(config)#interface g0/0/0
R1(config-if)#no shutdown
R1(config-if)#interface g0/0/0.10
R1(config-subif)#encapsulation dot1Q 10
R1(config-subif)#ip address 192.168.10.1 255.255.255.0
R1(config-subif)#interface g0/0/0.20

R1(config-subif)#encapsulation dot1Q 20
R1(config-subif)#ip address 192.168.20.1 255.255.255.0
R1(config-subif)#end
R1#
%SYS-5-CONFIG_I: Configured from console by console

R1#write memory
Building configuration...
[OK]
```

Step 2 — Switch Configuration

```
None
enable
configure terminal
hostname S1

vlan 10
name CLIENTS

vlan 20
name SERVERS

interface g0/1
switchport mode trunk

interface fa0/1
switchport mode access
switchport access vlan 10

interface fa0/2
switchport mode access
switchport access vlan 20

end
write memory
```

```
S1#enable
S1#configure terminal
Enter configuration commands, one per line.  End with CNTL/Z.
S1(config)#hostname S1
S1(config)#vlan 10
S1(config-vlan)#name CLIENTS
S1(config-vlan)#
S1(config-vlan)#vlan 20
S1(config-vlan)#name SERVERS
S1(config-vlan)#interface fa0/1
S1(config-if)#switchport mode access
S1(config-if)#switchport access vlan 10
S1(config-if)#
S1(config-if)#interface fa0/2
S1(config-if)#switchport mode access
S1(config-if)#switchport access vlan 20
S1(config-if)#end
S1#write memory
Building configuration...
[OK]
S1#
%SYS-5-CONFIG_I: Configured from console by console
```

Step 3 — Configure Hosts

Set static IP addresses per table.

🔍 Part 2 — Verify Connectivity

From PC1:

```
None
ping 192.168.20.20
```

-
```
C:\>ping 192.168.20.20

Pinging 192.168.20.20 with 32 bytes of data:

Reply from 192.168.20.20: bytes=32 time<1ms TTL=127
Reply from 192.168.20.20: bytes=32 time<1ms TTL=127
Reply from 192.168.20.20: bytes=32 time<1ms TTL=127
Reply from 192.168.20.20: bytes=32 time<1ms TTL=127

Ping statistics for 192.168.20.20:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 0ms, Maximum = 0ms, Average = 0ms

C:\>arp -a
  Internet Address      Physical Address      Type
  192.168.10.1          0090.2157.2701        dynamic
```

Verify:

- Successful replies
- ARP entries

Router checks:

```
None
show ip route
show ip interface brief
```

```
R1>show ip route
Codes: L - local, C - connected, S - static, R - RIP, M - mobile, B - BGP
       D - EIGRP, EX - EIGRP external, O - OSPF, IA - OSPF inter area
       N1 - OSPF NSSA external type 1, N2 - OSPF NSSA external type 2
       E1 - OSPF external type 1, E2 - OSPF external type 2, E - EGP
       i - IS-IS, L1 - IS-IS level-1, L2 - IS-IS level-2, ia - IS-IS inter area
       * - candidate default, U - per-user static route, o - ODR
       P - periodic downloaded static route

Gateway of last resort is not set

     192.168.10.0/24 is variably subnetted, 2 subnets, 2 masks
C       192.168.10.0/24 is directly connected, GigabitEthernet0/0.10
L       192.168.10.1/32 is directly connected, GigabitEthernet0/0.10
     192.168.20.0/24 is variably subnetted, 2 subnets, 2 masks
C       192.168.20.0/24 is directly connected, GigabitEthernet0/0.20
L       192.168.20.1/32 is directly connected, GigabitEthernet0/0.20

R1>show ip interface brief
Interface              IP-Address      OK? Method Status                Protocol
GigabitEthernet0/0     unassigned      YES unset  up                    up
GigabitEthernet0/0.10  192.168.10.1    YES manual up                    up
GigabitEthernet0/0.20  192.168.20.1    YES manual up                    up
GigabitEthernet0/1     unassigned      YES unset  administratively down down
Vlan1                  unassigned      YES unset  administratively down down
```

Switch checks:

```
None
show interfaces trunk
show mac address-table
```

```
S1>show interfaces trunk
Port        Mode         Encapsulation  Status        Native vlan
Gig0/1      on           802.1q         trunking      1

Port        Vlans allowed on trunk
Gig0/1      1-1005

Port        Vlans allowed and active in management domain
Gig0/1      1,10,20

Port        Vlans in spanning tree forwarding state and not pruned
Gig0/1      1,10,20

S1>show mac address-table
          Mac Address Table
-------------------------------------------

Vlan    Mac Address       Type        Ports
----    -----------       --------    -----

   1    0090.2157.2701    DYNAMIC     Gig0/1
  10    0090.2157.2701    DYNAMIC     Gig0/1
  20    0090.2157.2701    DYNAMIC     Gig0/1
```

🧪 Part 3 — Deploy MIME Server

On Server (PC2)

```
None
cd automation
python server/main_enhanced.py --host 0.0.0.0 --port 65432 --verbose
```

On Client (PC1)

```
None
cd automation
python client/main_enhanced.py assets/sample.png --host 192.168.20.20 --port
65432
```

Confirm file saved.

📡 Part 4 — Wireshark Capture Walkthrough on PC1 and PC1

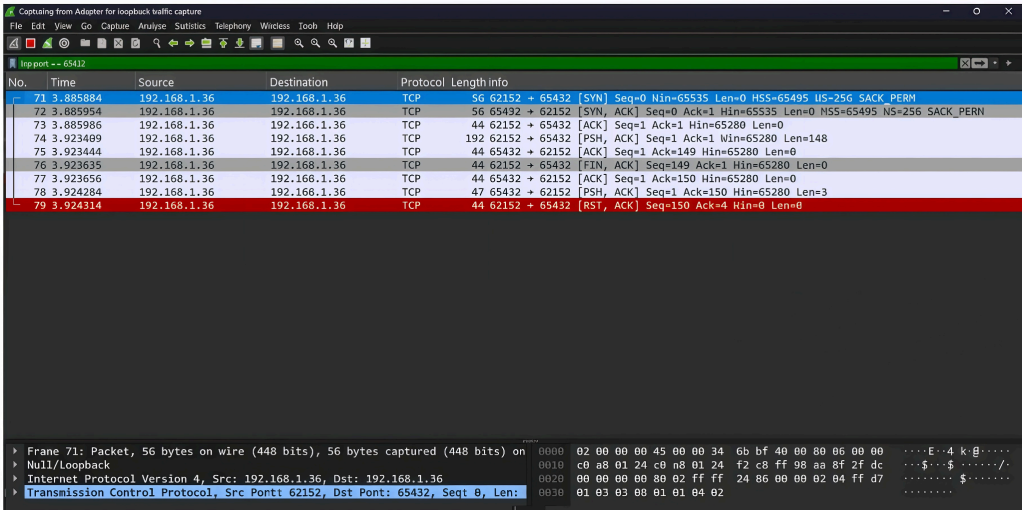
Step 1 — Start Capture

Open Wireshark.
Select interface connected to network.
Apply capture filter:

```
None
tcp port 65432
```

Step 2 — Initiate Transfer

Send file again.
Stop capture after completion.



Step 3 — Analyze Packets

A — ARP Exchange

Look for:

- Who has 192.168.20.20?

Confirm MAC resolution.

เนื่องจากเป็นการจับแพ็กเก็ตบน Loopback (127.0.0.1) การสื่อสารเกิดขึ้นภายในเครื่องเดียวกัน ข้อมูลจึงไม่ออกสู่เครือข่ายจริง ทำให้ไม่ต้องมีการค้นหา MAC Address และไม่เกิด ARP ขึ้นในการทดลองนี้

B — TCP Three-Way Handshake

Identify:

1 SYN 2 SYN-ACK 3 ACK

Check sequence numbers.

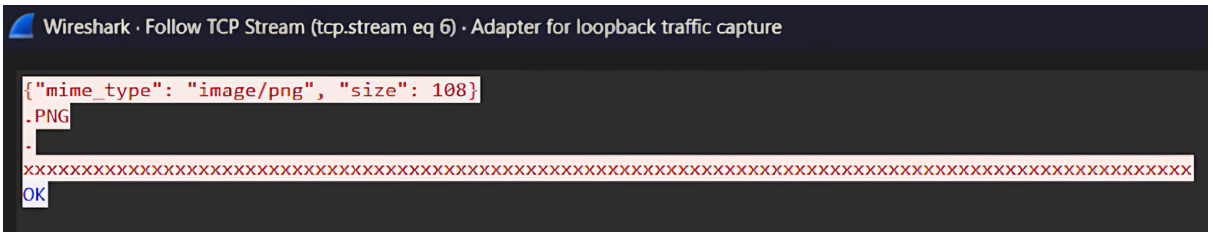
Protocol	Length	Info
TCP	56	57365 → 65432 [SYN] Seq=0 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM
TCP	56	65432 → 57365 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM
TCP	44	57365 → 65432 [ACK] Seq=1 Ack=1 Win=65280 Len=0

C — Data Transfer

Follow stream:
Right click → Follow → TCP Stream.
You should see JSON header similar to:

```
None
{"mime_type":"image/png","size":12345}
```

Followed by binary.


Wireshark · Follow TCP Stream (tcp.stream eq 6) · Adapter for loopback traffic capture
{"mime_type": "image/png", "size": 108}
.PNG
.
xxxx

🔬 Packet Dissection Guide

Layer	Field	What to Observe
Ethernet	Src/Dst MAC	Frame direction
802.1Q	VLAN ID	Segmentation
IP	TTL	Routing behavior
TCP	Flags	Reliability
Payload	JSON	MIME metadata

🛠️ Part 5 — Troubleshooting Labs

Scenario 1 — Broken Routing

Shutdown subinterface.

Observe:

- Ping failure
- TCP timeout

```
C:\>ping 192.168.20.20

Pinging 192.168.20.20 with 32 bytes of data:

Request timed out.
Request timed out.
Request timed out.
Request timed out.

Ping statistics for 192.168.20.20:
    Packets: Sent = 4, Received = 0, Lost = 4 (100% loss),
```

เมื่อปิดการทำงานของ subinterface เส้นทางระหว่าง VLAN จะถูกตัดออกทันที ทำให้อุปกรณ์ไม่สามารถส่งข้อมูลข้ามเครือข่ายได้ ผลที่เห็นคือ Ping ไม่ได้รับการตอบกลับ และการเชื่อมต่อ TCP ไม่สามารถสร้างได้จนเกิด timeout

สรุป: การ shutdown subinterface ทำให้การทำ Routing หยุดทำงาน จึงไม่สามารถติดต่อปลายทางได้ทั้งในระดับ ICMP และ TCP

Scenario 2 — Firewall Block

Block port.

Observe:

- SYN retransmissions

เมื่อทำการบล็อกพอร์ตปลายทางด้วย Firewall แพ็กเก็ต SYN ที่ไคลเอนต์ส่งไปจะไม่ได้รับการตอบกลับ ทำให้ไม่เกิด SYN-ACK ตามขั้นตอนของ Three-Way Handshake ใน Wireshark จะเห็นการส่ง SYN ซ้ำหลายครั้ง (SYN retransmissions) เพราะ TCP พยายามเริ่มการเชื่อมต่อใหม่ เมื่อยังไม่ได้รับการตอบสนอง สุดท้ายการเชื่อมต่อจะล้มเหลวและเกิด timeout

สรุป: การบล็อกพอร์ตทำให้ TCP ไม่สามารถสร้างการเชื่อมต่อได้ จึงเกิดการส่ง SYN ซ้ำเพื่อพยายามติดต่อปลายทาง แต่ไม่สำเร็จ

Scenario 3 — Wrong Gateway

Observe:

- ARP incomplete

```
C:\>ping 192.168.20.20

Pinging 192.168.20.20 with 32 bytes of data:

Request timed out.
Request timed out.
Request timed out.
Request timed out.

Ping statistics for 192.168.20.20:
    Packets: Sent = 4, Received = 0, Lost = 4 (100% loss),

C:\>arp -a
No ARP Entries Found
```

Scenario 4 — Packet Loss Simulation

Drop packets.

Observe:

- TCP retransmissions
- Window adjustment

ในกรณีใช้ Localhost จะไม่เกิด Packet Loss จริง เพราะข้อมูลไม่ผ่านเครือข่ายภายนอก แต่หากมีการจำลองการสูญหายของแพ็กเก็ต TCP จะตรวจจับได้จากการไม่ได้รับ ACK ตามกำหนด จากนั้นจะทำการส่งข้อมูลซ้ำ (retransmission) และลดขนาดหน้าต่างการส่งข้อมูลลงชั่วคราวเพื่อลดอัตราการส่ง เมื่อสถานการณ์ดีขึ้น TCP จะค่อย ๆ ปรับหน้าต่างให้เพิ่มขึ้นอีกครั้ง แสดงถึงกลไกความน่าเชื่อถือของโปรโตคอล TCP ในการควบคุมการรับส่งข้อมูล

🎓 Assessment Questions

1. Why does TCP retransmit lost segments?
   เพราะ TCP เป็นโปรโตคอลแบบ Reliable หากส่งข้อมูลไปแล้วไม่ได้รับ ACK ภายในเวลาที่กำหนด จะถือว่าข้อมูลสูญหายและส่งซ้ำ (retransmission) เพื่อให้ข้อมูลถึงปลายทางครบถ้วน
2. How does VLAN tagging affect broadcast domains?
   VLAN Tag (802.1Q) ใช้แยกเครือข่ายออกเป็นหลาย Broadcast Domain แม้ใช้สวิตช์ตัวเดียวกัน อุปกรณ์ใน VLAN หนึ่งจะไม่รับ Broadcast จากอีก VLAN หนึ่ง ช่วยลดการกระจายข้อมูลที่ไม่จำเป็น
3. What role does MIME play compared to TCP headers?
   TCP Header ทำหน้าที่ควบคุมการส่งข้อมูล (port, sequence, reliability) ส่วน MIME ทำหน้าที่บอกประเภทและความหมายของข้อมูล เช่น image/png จึงเป็นหน้าที่ระดับ Application ไม่ใช่การควบคุมการส่ง
4. Where does the routing decision occur?
   การตัดสินใจ Routing เกิดที่ Router ใน Layer 3 โดยดูจาก Destination IP Address แล้วเลือกเส้นทางจาก Routing Table
5. Why does ARP precede TCP communication?
   ก่อนส่ง TCP Segment ต้องรู้ MAC Address ปลายทางก่อน เพราะการส่งจริงเกิดใน Layer 2 ดังนั้น ARP จะทำงานเพื่อแปลง IP ไป MAC ก่อนเริ่มการสื่อสาร

📊 Reflection Exercise

Explain:

How does application meaning survive multiple encapsulation layers?

แม้ข้อมูลจะถูกห่อหุ้มหลายชั้น (TCP, IP, Ethernet) แต่แต่ละ Layer จะเพียงแค่เพิ่ม Header ของตัวเองโดยไม่แก้ไข Payload เดิม เมื่อถึงปลายทางจะถูกแกะออกทีละชั้น (Decapsulation) จนเหลือข้อมูล Application เดิมครบถ้วน ดังนั้นความหมายของข้อมูลจึงยังคงอยู่แม้ผ่านหลาย Layer

---

🚀 Extension Activities

- Add TLS encryption

  เมื่อเพิ่ม TLS การรับส่งข้อมูลถูกเข้ารหัส ทำให้ไม่สามารถอ่าน JSON Header หรือข้อมูลไฟล์ได้โดยตรงใน Wireshark เห็นเพียงข้อมูลที่ถูกเข้ารหัสเท่านั้น ส่งผลให้ความปลอดภัยเพิ่มขึ้น แต่มีขั้นตอน Handshake เพิ่มขึ้นก่อนเริ่มส่งข้อมูล
- Measure throughput vs latency

  เมื่อวัดค่า Throughput พบว่าการส่งไฟล์ขนาดใหญ่มีอัตราการรับส่งข้อมูลสูงกว่า แต่ใช้เวลานานขึ้นหาก Latency สูง ขณะที่ Latency ต่ำช่วยให้การตอบสนองเร็วขึ้น โดยเฉพาะช่วงเริ่มต้นการเชื่อมต่อ
- Capture checksum errors

  จากการตรวจสอบไม่พบ Checksum Error ในสภาวะปกติ แสดงว่าเฟรมและแพ็กเก็ตถูกส่งครบถ้วน หากมีความผิดพลาด Wireshark จะแสดงสถานะ Checksum incorrect
- Add checksum field to protocol

  เมื่อเพิ่มฟิลด์ Checksum ใน JSON Header สามารถตรวจสอบความถูกต้องของไฟล์หลังรับเสร็จได้ หากค่าไม่ตรงกัน แสดงว่าข้อมูลเสียหาย เพิ่มความน่าเชื่อถือในระดับ Application
- Containerize server

  เมื่อรัน Server ใน Container การติดตั้งและย้ายระบบทำได้สะดวกขึ้น ลดปัญหาความต่างของสภาพแวดล้อม และสามารถเปิดพอร์ตให้เข้าถึงได้เหมือนระบบจริง
- Compare with HTTP transfer

  เมื่อเปรียบเทียบกับ HTTP พบว่า HTTP มีโครงสร้าง Header มาตรฐานและรองรับการใช้งานผ่าน Browser ได้ง่ายกว่า ขณะที่โปรโตคอลที่พัฒนาเองมีความยืดหยุ่นสูงและเรียบง่ายกว่า แต่ต้องจัดการโครงสร้างเองทั้งหมด